Big 4 Thought-Leadership Intelligence

Project Report

An end-to-end Python pipeline that collects, cleans and analyses the published thought leadership of Deloitte, EY, KPMG and PwC to show what each firm emphasises.

Prepared by	Shayan Ponnanna Malchira
Input	INPUT_CSV.csv (200 papers)

1. Executive summary

The original work consisted of three separate scripts (download, extract, analyse) that depended on each other through file-name conventions and an import between the first and last script. This project restructures them into one installable package with a command-line interface, shared configuration, run manifests, automated tests and an automatically generated report. During the restructuring 16 defects or weaknesses were fixed (Section 6); the most important is that 16 of the 200 source URLs are PDF files, which the original downloader would have saved as corrupted HTML.

- **Stage 1 - scrape:** requests first, headless Chromium (Playwright) only for pages that look empty; HTML and PDF handled.
- **Stage 2 - extract:** trafilatura for article text and tables, BeautifulSoup and pypdf as fallbacks; pages under 150 words flagged.
- **Stage 3 - analyze:** TF-IDF distinctive terms, five keyword-lexicon themes per 1,000 words, readability, firm similarity, and a validation of the lexicons against the human category labels.
- **Stage 4 - report:** this PDF, rebuilt from the data every run.

2. Problem statement and objectives

Professional-services firms use published research to shape client agendas. Comparing four firms by hand across five topics and 200 documents is slow and subjective. The objectives are to (1) build a reproducible corpus from a curated URL list, (2) quantify which themes each firm stresses, (3) surface vocabulary that is distinctive to each firm, (4) describe how large and readable each firm's corpus is, and (5) make every step transparent enough to audit.

3. Input data

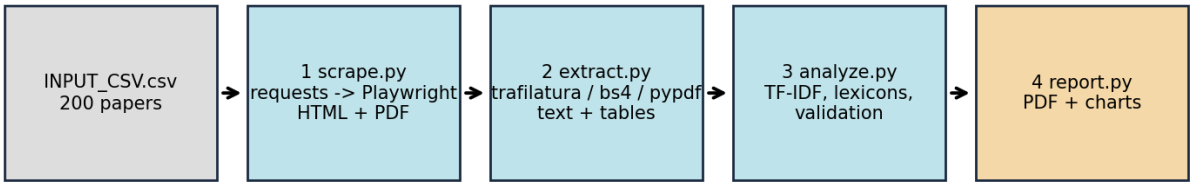
The input file lists 200 papers with the columns Firm, Title, Url, Publish_date and Trend_category. The table shows the papers per firm and category as given in the CSV (before any download).

firm	AI & Agentic	Finance & Work	Cyber & Risk	M&A & Deals	ESG & Climate	Total
Deloitte	10	10	10	10	10	50
EY	10	10	10	10	10	50
KPMG	10	10	10	10	10	50
PWC	10	10	10	10	10	50

Data-quality observations computed from the CSV:

- 16 URLs end in .pdf; 1 URL appears twice (different titles).
- 46 rows have no usable date ('n.d.', 'Not specified', 'Rolling/current'); the rest use at least 8 different date formats.
- Hosts: www.pwc.com (50), www.ey.com (50), www.deloitte.com (46), kpmg.com (44), assets.kpmg.com (6), www2.deloitte.com (2), www.investmentbanking.deloitte.com (2).

4. System design



manifests (scrape_manifest.csv, extraction_manifest.csv) connect the stages; papers.py + config.py are shared by all

Figure 1 - pipeline architecture

Module	Responsibility	Replaces
btli/config.py	Paths, thresholds, theme lexicons, stop-lists	constants scattered in 3 scripts
btli/papers.py	CSV loading, date parsing, unique file stems	build_output_path in scrape_whitepapers.py
btli/scrape.py	Download HTML/PDF with retries and browser fallback	scrape_whitepapers.py
btli/extract.py	Text, tables, PDF text, fallbacks, flags	extract_html.py
btli/analyze.py	Statistics, TF-IDF, theme density, validation, charts	analyze_themes.py
btli/report.py	PDF report generation	(new)
btli/demo.py	Synthetic offline corpus for demos and tests	(new)
btli/__main__.py	CLI: scrape | extract | analyze | report | all | demo	(new)

5. Methodology

5.1 Collection

Each URL is requested with a browser-like User-Agent. A page whose visible text is under 400 characters is treated as a JavaScript shell and re-fetched with Chromium. A random 2-3 s pause separates rows so the sites are not hammered. Files are cached, so a stopped run can be resumed. Rows that fail or stay thin are listed in failed_saves.csv.

5.2 Extraction

trafilatura scores page blocks and keeps article content, dropping menus, footers and cookie banners. Tables are taken from its XML output and written as CSV. If the result has fewer than 150 words a BeautifulSoup extraction is tried and the longer text is kept. PDFs are read with pypdf. Papers under the word threshold are flagged and excluded from the analysis.

5.3 Analysis

- **Corpus statistics:** papers, words, mean sentence length and Flesch reading ease per firm (syllables estimated from vowel groups).
- **Distinctive terms:** all of a firm's papers are joined into one document; uni- and bi-gram TF-IDF (sublinear tf) is computed across the four firm documents. Terms present in all four firms and publisher/site words are excluded.
- **Theme emphasis:** five lexicons, one per CSV category. Whole-word, case-insensitive matching with longest-term-first so there is no double counting. Reported as hits per 1,000 words.

- **Lexicon validation:** per paper the five densities are z-scored across the corpus and the highest is compared with the human Trend_category. Agreement is a sanity check of the lexicons, not a claim about the firms.
- **Firm similarity:** cosine similarity between the four firm-level TF-IDF vectors.

6. Defects found in the original scripts and how they were fixed

Area	Problem	Fix in this project
Download	16 of the 200 URLs are PDFs; they were saved through response.text as corrupt '.html'.	Detect PDF by Content-Type / %PDF magic bytes; save as .pdf; extract with pypdf.
Download	INPUT_CSV was defined twice ('csv_csv.csv' then 'INPUT_CSV.csv'); fragile order-dependent config.	Single source of truth in config.py + --csv option.
Download	A new Chromium instance was launched for every fallback page (slow, memory heavy).	One lazily-started browser/context reused for the whole run and closed in a finally block.
Download	Playwright waited only for 'networkidle' - analytics-heavy sites often never reach it and time out.	Wait for domcontentloaded, then best-effort networkidle (10 s cap).
Download	No retry/back-off; a transient 429/503 became a permanent failure. 403/404 were also fetched twice.	3 attempts with exponential back-off for 429/5xx/network errors; 4xx stops immediately.
Download	response.text can mis-decode pages with no charset header (ISO-8859-1 default).	Fall back to apparent_encoding when no charset is declared.
Download	Only failures were logged, so coverage could not be computed; cached files lost their status.	scrape_manifest.csv with status/method/kind/visible_chars for every row; resume-safe.
Naming	{firm}_{date}_{first 8 title words} could collide; the 'already saved' check then skipped a real paper. Dates such as 'Q4 2025', '2025/2026', 'n.d.' were mis-parsed by dateutil fuzzy mode.	6-char hash of firm+title+url added; explicit parser for quarter/range/no-date tokens; missing day/month never invented.
Extraction	Folders had to be passed by hand (python extract_html.py folderA folderB ...) and the path layout (extracted_text/<folder>/) only matched analysis by convention.	Driven by the paper list and the shared Paper.stem; no manual folder arguments.
Extraction	The BeautifulSoup fallback existed only as a comment.	Implemented: used when trafilatura gives nothing or fewer than 150 words and bs4 finds more.
Extraction	Malformed XML from trafilatura would crash the whole run.	ParseError handled per page; error recorded in extraction_manifest.csv.
Analysis	analyze_themes.py imported scrape_whitepapers just for build_output_path, pulling in requests/bs4 and breaking if Playwright config changed.	Shared papers.py; each stage can be run and tested alone.
Analysis	Lexicon keys ('AI & Agentic', ...) did not match the CSV categories, so lexicons could not be validated.	Keys now equal Trend_category; new lexicon_validation step reports agreement with human labels.
Analysis	Overlapping terms were double-counted ('generative ai' and 'ai' both hit).	Single longest-first alternation regex per theme.
Analysis	TF-IDF with max_df=1.0 and publisher names left in the text let site boilerplate and firm names dominate 'distinctive' terms.	max_df=0.75 (term must be absent from at least one firm) and a boilerplate stop-list.
Analysis	Only 3 outputs (stats, terms, density).	Added readability (Flesch), category coverage, firm similarity, lexicon validation and 5 charts.

7. Results

Results basis: Synthetic demo corpus (offline validation - not real firm content)

7.1 Collection coverage

download status	papers
saved	192
thin	8

Of 200 papers, **192** have at least 150 words of extracted text and enter the analysis; 8 are flagged for manual review (see extraction_manifest.csv).

7.2 Corpus size and readability

firm	papers	total_words	avg_words	avg_sentence_len	flesch_reading_ease
Deloitte	48	35360	736.7	13.2	-11.2
EY	48	35464	738.8	13.1	-12.5
KPMG	48	32613	679.4	13.3	-13.5
PWC	48	34246	713.5	13.2	-11.2

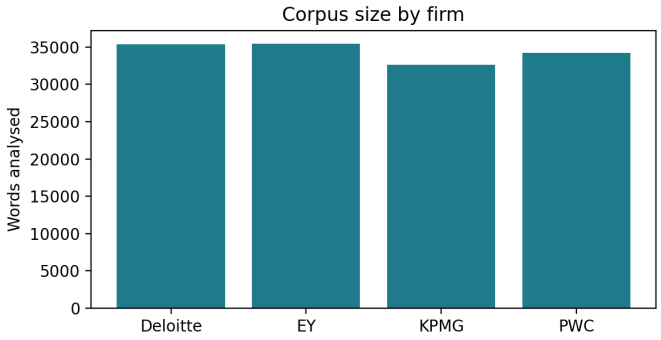


Figure 2 - words analysed per firm

7.3 Theme emphasis

Each cell is the number of lexicon hits per 1,000 words in a firm's usable papers. Compare across a row to see which theme a firm leans towards; compare down a column to see who stresses a theme most.

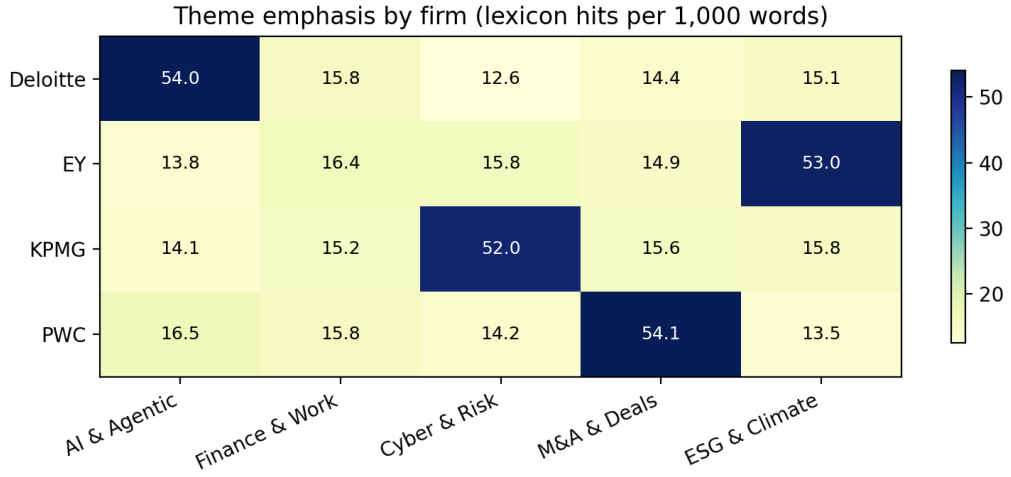


Figure 3 - theme emphasis by firm

Highest-density theme per firm: **Deloitte** - AI & Agentic (53.96); **EY** - ESG & Climate (52.98); **KPMG** - Cyber & Risk (52.0); **PWC** - M&A & Deals (54.08).

7.4 Distinctive vocabulary

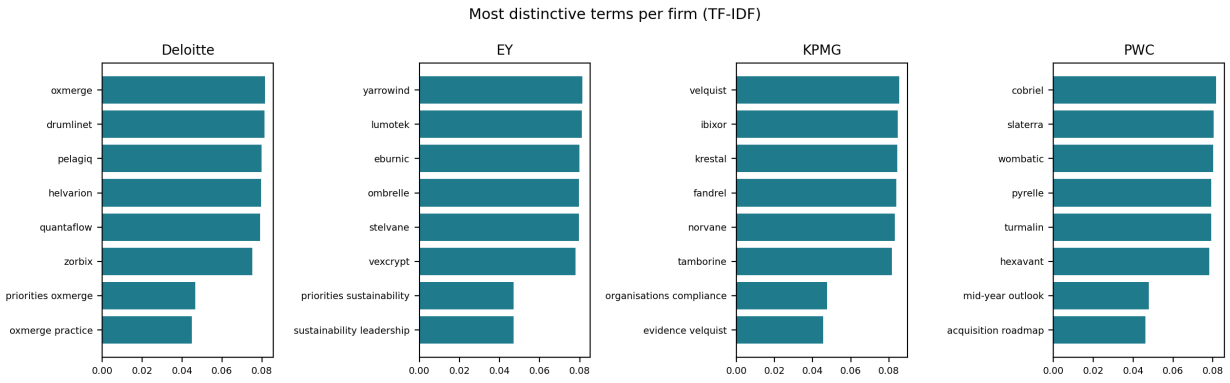


Figure 4 - top TF-IDF terms per firm

firm	top 5 distinctive terms
Deloitte	oxmerge, drumlinet, pelagiq, helvarion, quantaflow
EY	yarrowind, lumotek, eburnic, ombrelle, stelvane
KPMG	velquist, ibixor, krestal, fandrel, norvane
PWC	cobriel, slaterra, wombatic, pyrelle, turmalin

Planted-signal recovery test (synthetic run only): invented marker words were inserted into each firm's texts; a working TF-IDF step must rank them for the right firm.

Firm	Planted marker words	Found in top 15	Recovered
Deloitte	6	6	100%
KPMG	6	6	100%
PWC	6	6	100%
EY	6	6	100%

7.5 Firm similarity

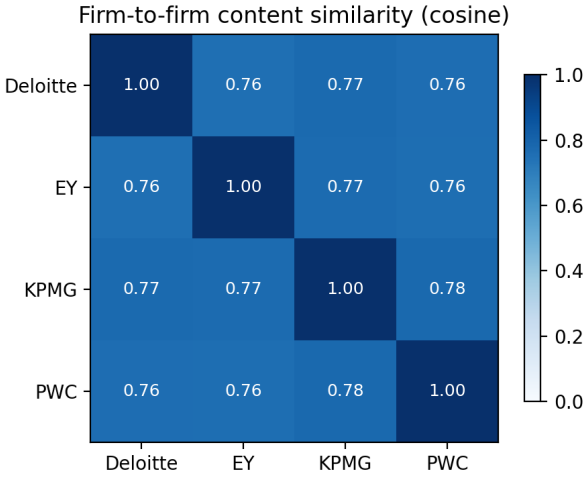


Figure 5 - cosine similarity of firm corpora

7.6 Lexicon validation

Across 192 usable papers the lexicon-based prediction matches the human category label for **84.4%** of papers (random guessing among five classes would give about 20%).

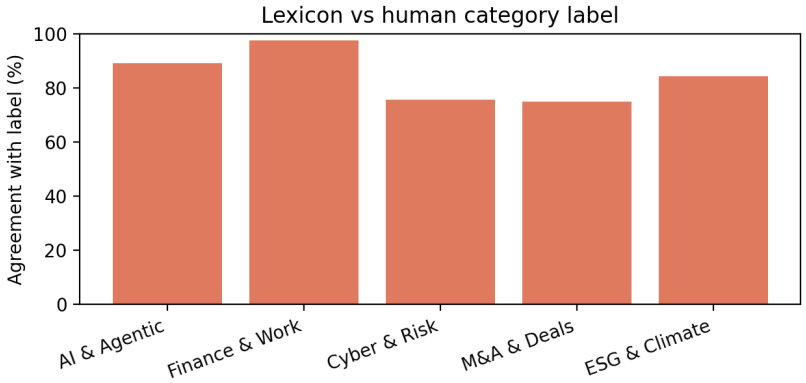


Figure 6 - agreement by category

8. Testing

The tests/ folder holds 13 automated pytest tests. They cover date parsing for every format found in the CSV, unique file names for all 200 rows, the no-double-counting rule of the lexicon matcher, HTML/table/PDF extraction, thin-page flagging, and a full offline pipeline run (generate synthetic corpus, extract, analyse, build PDF) including the planted-signal recovery check. Run them with `pytest -q`. Network behaviour (live downloading) cannot be unit-tested deterministically and must be checked by running stage 1 on a small sample (`python -m btli scrape --limit 5`).

9. Limitations and responsible use

- **Only four documents enter TF-IDF** (one per firm). It highlights vocabulary, but small differences are not statistically tested.
- **Keyword lexicons are a proxy** for topics: they miss paraphrase and can count a word used in another sense ('deal with'). The validation step quantifies this but does not remove it.
- **Selection bias:** the 200 URLs were curated (50 per firm, 40 per category), so results describe that sample, not each firm's entire output. Landing pages and press releases are mixed with full reports.
- **Balanced design:** 10 papers per firm and category means theme densities mostly measure how heavily each firm uses a topic's vocabulary, not how often it chooses to write on the topic.
- **Dates are descriptive only:** about one-fifth of rows have no usable date, so no time-trend analysis is made.
- **Access:** some sites block automated clients, require JavaScript, or sit behind consent walls; those pages appear in failed_saves.csv. Check each site's terms of use and robots.txt before running at scale, keep the 2-3 s delay, and do not redistribute the downloaded content.
- **Language:** English-only stop-words, readability formula and lexicons; non-English pages will be mis-measured.

10. Environment and how to run

Item	Value
Python	3.10 - 3.12
Libraries	requests, beautifulsoup4, lxml, trafilatura, pypdf, python-dateutil, pandas, numpy, scikit-learn, matplotlib, reportlab, pillow, playwright, pytest
Browser	Chromium via 'playwright install chromium' (only used as fallback)
Install (pip)	python -m venv .venv; activate; pip install -r requirements.txt; playwright install chromium
Install (conda)	conda env create -f environment.yml; conda activate btli; playwright install chromium
Full run	python -m btli all (expect roughly 10-30 min: 200 rows x 2-3 s delay plus download time)
Offline demo	python -m btli demo (synthetic data, runs in seconds)
Single stages	python -m btli scrape | extract | analyze | report [--limit N] [--no-playwright] [--workdir DIR]
Docker	docker build -t btli . && docker run --rm -v $PWD/output:/app/output btli

11. Future work

- Replace keyword lexicons with embedding- or LLM-based topic classification and compare with the human labels.
- Paper-level (not firm-level) TF-IDF or BERTopic to find sub-themes inside each category.
- Add sentiment / hedging language analysis and a time-trend view once dates are researched manually.
- Respect robots.txt automatically and add conditional GET (ETag) so re-runs only fetch changed pages.
- Interactive dashboard (Streamlit) on top of the analysis CSVs.